

Q1. The KMP algorithm can be modified by keeping note of the current number of accurate matches. This value will be a note of the largest possible match so far, which is also the largest prefix of the substring. The KMP algorithm watches the matching elements of a substring and a string up to a point. Making a variable called MAX_MATCH that you can use to compare to each time a new letter is judged will yield the largest prefix of a String P that matches the substring T.

Only those proper prefixes that are larger in length than the variable MAX_MATCH will be considered, if no other prefixes are larger than it, then the current prefix must be the largest and that is the one that can be returned

This change to the original algorithm should add an extra step, comparing the length of the current match to the variable MAX_MATCH which should only add $O(1)$ time to the complexity, leaving its complexity at $O(n + m)$

Q2. Using the Print-LCS algorithm, because it has a complexity of $\Theta(m + n)$, it can be altered so that instead of taking in the b table and the lengths of X and Y, and X and Y, it takes in the c table, and X and Y. This would end up printing the longest common subsequence of X and Y. This version of the algorithm would compute the b table values as it occurs, not storing and referring back to them. This would occur as the algorithm iterates through each of the strings to find commonalities. In this version of the Print-LCS algorithm, because the c table entry depends on if the current $x_i = y_i$ values, this would lead to printing them as they appear, and not by iterating through the b table after the fact. This should leave the complexity at $\Theta(m + n)$ because no additional comparisons per element are needed.

Q3. Following methods of greedy algorithms, only the globally optimal solutions are considered. This is done by making locally optimal choices, considering the choice that looks best as the current problem, without considering results from subproblems. In this particular problem, the best choice would be to make professor Gekko travel as much as he can while being able to get water from one of the water stops before he runs out. This stop must be less than m miles from professor Gekko's starting point, meaning that from a point called start, there is optimal paths (subproblems that have all been solved).

To prove this plan, you can look at possible end points called $s_1, s_2, \dots$, and then you can look at some points he can go to from his starting point without running out of water, $n_1, n_2, \dots$. These stop points will always be less than the maximum distances that can be travelled, $s_1 < n_1, s_2 < n_2$, etc... meaning that no matter what point you look at, using the greedy algorithms would mean that this plan of action for professor Gekko's trip is optimal.

Q4. The value $2n-1$ holds significance for a complete binary tree as it would have $2n-1$ nodes, which can be determined through traversal. We know that any character of the set C can be encoded in $\lceil \log n \rceil$ bits which is where this form of the equation comes from. Because there are n characters inside of the encoding (n letters to be encoded from set C), they can be encoded using $n \lceil \log n \rceil$ bits. Combined with the complete traversal of the binary tree, it would yield $2n-1 + n \lceil \log n \rceil$.

Q5. Keeping not of all the spaces between words is important for this algorithms success. A variable should be able to store all extra spaces between words, noting its location between a word w_i and word w_j . Given these spaces, after the first iteration through all the words, then the cost for keeping words w_i and w_j in the same line can be stored in another variable, also noting the positions of both words. After that iteration, the minimum cost based on the word placement in lines will be calculated and stored in another variable. After this the words can be organized to be in as many lines as deemed least costly. Because the first iteration to find spaces between words occurs over the M words, this section has $O(n)$ time complexity. After that, another set of operations must be performed, per division of n. Therefore, this algorithm would have time complexity $O(n^2)$.

Q6. A maximum spanning tree can be created in a similar way to a minimum spanning tree, by choosing the most expensive/highly weighted edge of the graph instead of the smallest. This would be done by making all the edges ($E = \{u, v\}$) of the graph go into a heap (max heap), each vertex (V) would go into a set by themselves (make_set(v)). After this algorithm would check that while a variable found is set to false (the maximum edge hasn't been found yet), delete the maximum edge $\{u, v\}$ from the heap:

if u and v are not in the same set (not part of the list of edges to traverse), then mark edge $\{u, v\}$ as a tree edge and union the sets with u and v

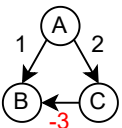
if u and v are in the same set, then do nothing

This maximum spanning tree algorithm will work as each step will only yield the largest edge of the graph, which will lead to the largest edges being traversed first.

This algorithm should have the same time complexity as the original minimum spanning tree algorithm as they both so the same type of steps, just for different variables. As such, their time complexity should also be the same, leaving the maximum spanning tree with a time complexity of $O((|V| + |E|) \log(|V|))$.

Q7. When following Dijkstra's algorithm, a minimum heap is used to note the vectors and edges, and will delete the smallest/minimum value from it. For a graph ($G = \{V, E\}$) containing negative weight edges, a distance value of anything other than a positive value will not be effectively compared and will lead to a minimum path that is longer that it should be.

A counter example would be: A graph, $G = \{V, E\}$, that contains $V = \{A, B, C\}$ and $E = \{1, 2, -3\}$ such that



For this example, the shortest distance from A to B should be A to C and then C to B because, based on their weights, $2 + -3 = -1$, but since the algorithm won't travel the additional distance, because it has already compared the weights from traveling from A to B and from A to C (here the weight of 1 is less than 2 so the path A to B is deemed the shortest path and is traversed) it would use the weight of 1 as the minimum distance instead.

Q8. Having a negative weight cycles introduces issues to weighted graphs when using the all-pair-shortest-pair algorithm. Just negative weights on the other hand could still work for a graph $G = (V, E)$, using the all-pair-shortest-pair algorithm. This is due to the fact that the all-pair-shortest-pair algorithm, aka Floyd-Warshall algorithm, is not dependent of the graphs edge weights, while Dijkstra's algorithm does. This algorithm considered all paths, where the shortest path between a vertex v_n and a vertex v_m are calculated and can be input into an adjacency matrix. This also allows the matrix to be updated every time a shorter path between the vertices is found. If there are no negative cycles in this graph, then there are no cycles which would usually cause the weight of the cycle to be 0 since the path is the shortest path as a vertex in the cycle is visited more than once (This would happen because the algorithm would remove all vertices between a start point and end point leaving this particular vertex v_i to visit itself in 0 steps).

The path from vertex v_n to v_m can go through many other vertices (v_p) but only takes the minimum value from the shortest path between v_n to v_p + v_m to v_p , leaving no dependency on non-negative edges as long as all possible paths are traversed to find the minimum one between two vertices. This would allow for just negative weighted edges to be functional in the all-pair-shortest-pair algorithm, but not the negative cycles.